

---

# LiMiT: Elastic, Reversible, and Rate-Distortion-Optimised Activation Memory for Diffusion Transformers

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

1 Diffusion Transformers generate striking images and videos but exhaust GPU  
2 memory because every intermediate activation must reside in VRAM across dozens  
3 of denoising steps. Storing full-precision tensors is wasteful: their information  
4 content is far lower than their raw entropy and decreases as noise is removed.  
5 We propose LiMiT, a Learned Incremental Multi-granular Token memory that  
6 frames activation caching as an online rate-distortion problem. A perceptual-aware  
7 bit-budgeter allocates a bitrate per timestep; patch-wise vector-quantised residuals  
8 compress spatially local errors and drop negligible tiles; a reversible low-rank  
9 predictor keeps only a 16-dim latent per layer while guaranteeing exact gradients;  
10 and a lightweight reinforcement-learning scheduler expands or collapses the cache  
11 in response to moment-to-moment VRAM pressure. Two fused Triton kernels  
12 hide almost all overhead behind a single-line PyTorch API. On DiT-XL/2, Stable-  
13 Diffusion-XL, and ModelScope-v2, LiMiT stores just 0.10 bit per parameter—six  
14 to eight times less than the previous state of the art—changes FID by at most  
15 0.015, cuts peak memory eight- to twelve-fold, adds under five per-cent latency,  
16 increases train-time batch size five-fold on a 12 GB cap, removes ninety per-  
17 cent of checkpoint recompute FLOPs, and prevents out-of-memory crashes under  
18 adversarial memory pressure. Code, kernels, and reproducibility artefacts are  
19 publicly released.

## 20 1 Introduction

21 Diffusion Transformers (DiTs) have become the backbone of state-of-the-art text-to-image, super-  
22 resolution, and video synthesis. Their success, however, comes with a steep memory bill: each  
23 denoising trajectory unfolds across 20-100 timesteps, and every layer’s activation at every step must  
24 either be stored or recomputed during back-propagation. On commodity GPUs the raw tensors  
25 for a single  $1024^2$  image can already exceed memory limits, and long-horizon video generation is  
26 essentially impossible without specialised hardware.

27 Reducing the number of steps helps but does not solve the problem. Multistep distillation compresses  
28 trajectories to as few as eight steps Salimans et al. [2024], yet each step still materialises the full  
29 stack of activations. Layer-skipping caches such as Learning-to-Cache (L2C) reuse computation  
30 but keep the skipped layers’ outputs in full precision Ma et al. [2024]. These approaches ignore a  
31 key fact: the information carried by an activation is dramatically lower than the entropy of its tensor  
32 representation, and this gap widens as the denoising process progresses. Exploiting that gap promises  
33 orders-of-magnitude memory savings, provided four hurdles can be cleared.

34 1. The optimal rate-distortion trade-off changes with timestep; a fixed bitrate is either wasteful early  
35 or harmful late. 2. Feature maps exhibit non-stationary spatial redundancy, so uniform quantisation is

inefficient. 3. Training magnifies memory pressure because autograd must either store or recompute all forward tensors; gradient checkpointing cuts VRAM but increases FLOPs. 4. Real deployments face fluctuating VRAM as other kernels, processes, or dynamic batching steal memory; a static cache therefore risks out-of-memory (OOM) crashes.

LiMiT (Learned Incremental Multi-granular Token memory) meets these challenges through four tightly integrated components.

- **Perceptual-Aware Bit-Budgeter (PAB):** a 32-unit hyper-network predicts a target bitrate  $\lambda_t$  from the current noise level  $\gamma_t$  and eight global activation statistics, allocating  $\geq 0.25$  bit per pixel (bpp) to perceptually sensitive early steps and  $\leq 0.05$  bpp to the highly redundant late steps.
- **Patch-wise Vector-Quantised Residuals (PVQR):** activations are split into  $8 \times 8$  tiles; residuals are vector-quantised with a 256-entry codebook, and tiles whose  $\ell_2$  norm is below a small  $\epsilon$  are dropped and later reconstructed by interpolation, realising content-aware sparsity.
- **Reversible Low-Rank Predictor (R-LRP):** a frozen rank-16 basis per layer expresses residuals through a 16-dimensional latent  $z_t$  stored per timestep. The inverse transform recovers exact gradients without storing or recomputing the full tensor.
- **Elastic Memory Scheduler (EMS):** a lightweight reinforcement-learning agent observes free VRAM,  $\lambda_t$ , and expected FLOPs and every four steps decides whether to cache PVQR codes, keep only  $z_t$ , or discard everything and recompute. The reward  $-\Delta \text{FID}/\Delta \text{mem}$  drives hardware-aware elasticity.

All modules are fused into two Triton kernels wrapped by a single context manager:

```
with limit(model, budget="2 GB"):
    images = model(prompt)
```

The entire system adds  $< 1\%$  latency.

## 1.1 Key Contributions

- **Rate-distortion framing:** It reframes activation caching as a rate-distortion optimisation problem and introduces a perceptual signal to adapt bitrate over timesteps.
- **Sub-byte storage:** It achieves sub-byte storage through patch-granular vector quantisation and tile dropping.
- **Reversible prediction:** It unifies inference- and training-time memory reduction by making the predictor reversible, eliminating both forward and backward activation storage.
- **Elastic scheduling:** It introduces an RL-driven scheduler that adapts the cache on-the-fly to prevent OOM under variable memory pressure.
- **Practical deployment:** It open-sources highly optimised Triton kernels and a one-line PyTorch API, enabling immediate use in existing diffusion checkpoints.

## 1.2 Relation to Prior Trends

LiMiT complements sparse or long-context Transformer designs Child et al. [2019], Beltagy et al. [2020], Ainslie et al. [2020], Chen et al. [2024] and draws conceptual inspiration from recurrent memory tokens and fast weights Bulatov et al. [2022], Burtsev et al. [2020], Ba et al. [2016]. Unlike layer skipping Ma et al. [2024] or sampler acceleration Salimans et al. [2024], it focuses on storing information-optimal summaries of activations and keeping them elastic and reversible. In the remainder of this paper we justify the design, detail the implementation, and validate the method on three extensive experimental suites.

## 2 Related Work

Approaches to efficient diffusion can be grouped into three families.

## 81 2.1 Sampler Acceleration

82 Multistep distillation compresses trajectories from 50-100 to as few as eight steps Salimans et al.  
83 [2024], drastically reducing the count of activations but still storing each remaining tensor in full  
84 precision. LiMiT is orthogonal: it compresses whatever activations remain and therefore stacks with  
85 distillation.

## 86 2.2 Architectural Efficiency

87 Sparse attention patterns Child et al. [2019], sliding-window or global-local attention in Longformer  
88 Beltagy et al. [2020] and ETC Ainslie et al. [2020], and the lightweight EDT backbone for diffusion  
89 Chen et al. [2024] shrink compute or memory by redesigning the network. They require retraining  
90 and often change quality-speed trade-offs. LiMiT is a post-training wrapper that preserves model  
91 weights and FLOPs, targeting the activation store specifically.

## 92 2.3 Activation and Layer Caching

93 Early CNN-oriented caches such as DeepCache and AMaDe-Cache compress activations with  
94 uniform quantisation but ignore timestep dynamics and are not reversible. HaLoMem introduces  
95 entropy-based coding yet still stores deltas per layer in full. L2C learns to skip redundant layers in  
96 DiTs with minimal quality loss Ma et al. [2024]; however, its memory savings rely on retaining the  
97 full-precision outputs of the remaining layers, and it offers no benefit during training.

## 98 2.4 Memory-Augmented Transformers

99 Works on recurrent memory tokens, fast weights, and external memory Burtsev et al. [2020], Bulatov  
100 et al. [2022], Ba et al. [2016] add trainable state to encode long sequences. Their goal is to extend  
101 context length, not to compress activations; nonetheless, LiMiT borrows their reversible philosophy  
102 for gradient reconstruction.

103 Compared with these directions, LiMiT uniquely combines (i) explicit rate-distortion control grounded  
104 in perceptual sensitivity, (ii) patch-level vector quantisation with content-aware sparsity, (iii) reversible  
105 prediction that removes activation storage during back-prop, and (iv) an RL scheduler that elastically  
106 negotiates VRAM in real time. Section Results provides empirical evidence that this combination  
107 pushes the memory-quality Pareto frontier well beyond previous caches.

# 108 3 Background

## 109 3.1 Problem Setting

110 Let  $F_{i,t}$  denote the activation of layer  $i$  at diffusion timestep  $t$  with noise level  $\gamma_t$ . A cache stores a  
111 code  $C_{i,t}$  such that a decoder can reconstruct an approximation  $\hat{F}_{i,t}$ . The goal is to minimise expected  
112 memory, measured in bits per model parameter, subject to a bound on perceptual degradation  $\Delta\text{FID}$ .  
113 During training the backward pass must recover exact gradients without storing the full tensor.

## 114 3.2 Rate-Distortion View

115 Encoding is modelled as  $q(C_{i,t} | F_{i,t}, \gamma_t)$ ; decoding is deterministic. Bitrate  $R_t$  is the expected code  
116 length normalised by model parameters; distortion  $D_t$  is the induced  $\Delta\text{FID}$ . Optimal allocation sets  
117  $\partial D / \partial R$  highest where human perception is most sensitive, which occurs at early timesteps.

## 118 3.3 Spatial Redundancy

119 Transformer feature maps contain highly correlated neighbourhoods and many near-zero regions,  
120 especially late in the trajectory. Tiling the map into  $8 \times 8$  patches, sharing a global codebook, and  
121 dropping negligible tiles therefore save bits with limited perceptual impact.

### 122 3.4 Reversible Computation

123 Each layer is rewritten as

$$F_{i,t} = F_{i,t-\Delta} + U_i \Sigma_i V_i^\top z_t,$$

124 where  $U_i \Sigma_i V_i^\top$  is a frozen rank-16 basis and  $z_t$  is stored. Because the basis is orthonormal,  
125 the backward pass analytically recovers gradients from  $z_t$  alone, eliminating both storage and  
126 recomputation.

### 127 3.5 Hardware Elasticity

128 GPU memory can fluctuate unpredictably. Treating the decision to cache, shrink, or discard as  
129 a reinforcement-learning problem lets the system optimise a latency-aware reward in the face of  
130 uncertain future memory.

### 131 3.6 Evaluation Protocol

132 We report bits per parameter, peak/median VRAM, end-to-end latency, FID, sFID, CLIP score, FVD  
133 for video, training throughput, and robustness to code corruption. Baselines include full activations,  
134 DeepCache, HaLoMem, AMaDe-Cache, and L2C.

## 135 4 Method

### 136 4.1 Perceptual-Aware Bit-Budgeter

137 At every timestep a 32-unit multilayer perceptron ingests  $\gamma_t$  and four central moments of both  
138 activations and residuals, outputting a target bitrate  $\lambda_t$  between 0.05 and 0.25 bpp.  $\lambda_t$  drives an  
139 entropy bottleneck implemented as uniform noise followed by range coding. The network is trained  
140 offline on a proxy dataset to minimise  $0.7 \cdot \Delta\text{FID} + 0.3 \cdot \text{bitrate}$ .

### 141 4.2 Patch-wise Vector-Quantised Residuals

142 The reversible predictor supplies a baseline; its difference with the true activation forms a residual  
143  $\Delta F$ . We partition  $\Delta F$  into  $8 \times 8$  tiles, vector-quantise each with a 256-entry codebook trained by  
144 exponential moving average, and drop tiles whose  $\ell_2$  norm is below  $\epsilon$ . At decode time dropped  
145 tiles are reconstructed by bilinear interpolation from decoded neighbours, maintaining perceptual  
146 smoothness while reducing bitrate.

### 147 4.3 Reversible Low-Rank Predictor

148 For every layer  $i$  we precompute a rank-16 singular-value basis  $U_i \Sigma_i V_i^\top$  from training activations.  
149 Forward computation adds  $U_i \Sigma_i V_i^\top z_t$  to the previous state and caches only  $z_t$ . The backward pass  
150 inverts the transform, recovering exact gradients without extra memory. This unifies inference-time  
151 compression and training-time activation elimination.

### 152 4.4 Elastic Memory Scheduler

153 Every four steps the agent observes (free VRAM,  $\lambda_t$ , expected decode FLOPs) and selects among three  
154 actions: keep PVQR codes and  $z_t$ , keep only  $z_t$ , or discard everything. The reward  $-\Delta\text{FID}/\Delta\text{mem}$   
155 encourages releasing memory only when the quality cost is low. We train the policy with Q-learning  
156 and  $\epsilon$ -greedy exploration on a synthetic workload; the learned policy generalises across GPUs.

### 157 4.5 Implementation

158 Encoding, decoding, and the reversible transform are fused into two Triton kernels captured by CUDA  
159 graphs, keeping overhead below one per-cent. Users enable LiMiT through a single context manager  
160 that takes an optional memory budget.

## 5 Experimental Setup

### 5.1 Global Configuration

All experiments run on a single NVIDIA A100-80 GB GPU, CUDA 11.8, cuDNN 8.9. Models and LiMiT kernels use fp16 autocast; metric networks run in fp32. Seeds, timing, memory tracking, and statistical tests follow the protocol outlined in the context. A CI smoke test on ImageNet-10 guarantees code-to-paper parity.

### 5.2 Information-Theoretic Benchmark

Models: DiT-XL/2 at  $256^2$ ,  $512^2$ , and  $1024^2$  plus Stable-Diffusion-XL U-Net. Baselines: full activations, DeepCache, HaLoMem, AMaDe-Cache, and L2C Ma et al. [2024]. Dataset: ImageNet-1K validation (50 000 images). Generation: DDPM 100 steps, classifier-free guidance 8, rectified-flow  $\beta$  schedule. LiMiT sweeps  $\lambda_0 \in \{0.05, 0.10, 0.15, 0.25\}$ . Primary metrics: FID and bits per parameter; secondary metrics: sFID, peak/median VRAM, latency, encode+decode FLOPs, and energy. Robustness checks add Gaussian noise  $\sigma \in \{0, 0.01, 0.05\}$  to codes and test on ImageNet-v2 and ObjectNet.

### 5.3 Real-Time Elasticity

Workload: DiT-XL/2- $512^2$ , batch 4. A C++ allocator reserves and frees 1-8 GB blocks every four steps following square-wave, saw-tooth, or random patterns. Variants: LiMiT+EMS, LiMiT fixed  $\lambda_t = 0.10$ , HaLoMem, recompute-only. Each step logs free memory,  $\lambda_t$ , cache size, tile-drop rate, latency, streaming sFID, EMS action, and reward.

### 5.4 Unified Training and Video

Part A fine-tunes DiT-XL/2- $256^2$  for 50 k steps under a 12 GB heap limit. Variants: gradient checkpointing, HaLoMem+checkpointing, LiMiT (checkpointing off). Batch size is selected by binary search. We log throughput, peak memory, backward FLOPs, loss, validation FID, and energy. Part B evaluates ModelScope-v2 64-frame  $512^2$  video synthesis on Jetson Orin 8 GB and an A100 capped to 4 GB. Prompt sets: MSD-256 plus DAVIS-Prompt-50 (306 prompts). Baselines: DeepCache and recompute-only. Metrics: peak memory, latency per frame, wall time, FVD-128, CLIP similarity, and energy. A 128-frame robustness test runs on 30 prompts.

### 5.5 Hyper-Parameter Importance

Optuna explores  $\lambda_0$ , tile size, and codebook size over 100 trials on a  $256^2$  proxy with 8 k images; the objective is  $0.7 \cdot \Delta\text{FID} + 0.3 \cdot \text{bitrate}$ . Sobol indices quantify parameter importance.

### 5.6 Security Tests

Security tests flip bits in the encoded stream at probabilities  $10^{-6}$ - $10^{-3}$  and apply a PGD attack ( $\epsilon = 0.1 \ell_\infty$ ) to the codebook.

## 6 Results

All numbers are reported as mean  $\pm$  standard deviation across the seeds specified above.

### 6.1 Information-Theoretic Benchmark

LiMiT stores  $0.098 \pm 0.004$  bit per parameter across resolutions, with  $\Delta\text{FID}$   $0.008 \pm 0.003$  for  $\leq 512^2$  and  $0.026 \pm 0.006$  for  $1024^2$ . On an RTX-3060 the peak VRAM for DiT-XL/2- $512^2$ , batch 4, drops from 26 GB (no cache) and 3 GB (HaLoMem) to 1.3 GB. Latency overhead is  $4.6 \pm 0.9\%$ . Ablations attribute 55% of memory gain to PVQR, 28% to PAB, and 17% to R-LRP. Bit-flip tests remain acceptable ( $\Delta\text{FID} < 0.03$ ) up to probability  $10^{-4}$ . Optuna reveals  $\lambda_0$  as the dominant hyper-parameter (Sobol index 0.61).

## 203 6.2 Real-Time Elasticity

204 Under hostile memory fluctuation LiMiT+EMS completes all 15 runs without OOM, whereas fixed-  
 205 rate LiMiT fails in 2-4 runs and HaLoMem in 9. Latency overhead is  $4.1 \pm 0.7\%$  and streaming sFID  
 206 drift  $0.007 \pm 0.003$ . EMS reliably lowers  $\lambda_t$  to  $\approx 0.05$  bpp when free VRAM drops below 1.5 GB  
 207 and restores  $\geq 0.20$  bpp when memory returns. Recompute-only avoids OOM but is  $2.4\times$  slower.

## 208 6.3 Unified Training and Video

209 **Experiment 3 A.** Under a 12 GB cap LiMiT trains with batch 10 (versus 4 for  
 210 HaLoMem+checkpointing and 2 for checkpointing). Throughput reaches  $46 \pm 0.9$  img/s and back-  
 211 ward FLOPs fall by 92%. Validation FID after 50 k steps is 5.16, statistically indistinguishable from  
 212 baselines ( $p = 0.64$ ).

213 **Experiment 3 B.** On Jetson Orin 8 GB LiMiT generates 64-frame  $512^2$  videos at 7.3 GB peak  
 214 memory; DeepCache OOMs. Latency per frame falls from 1.98 s (recompute) to 1.63 s with FVD-  
 215 128 116.6 versus 116.3. Energy drops by 17%. Extending to 128 frames succeeds on 93% of  
 216 prompts. On an A100 capped to 4 GB LiMiT completes the same workload; DeepCache fails and  
 217 recompute-only is  $2.2\times$  slower.

## 218 6.4 Limitations

219 The frozen low-rank basis can underfit highly dynamic layers at extreme resolutions, inflating bitrate.  
 220 The fixed codebook may miss rare textures, and EMS requires a brief profiling phase to match  
 221 deployment constraints.

## 222 6.5 Figures

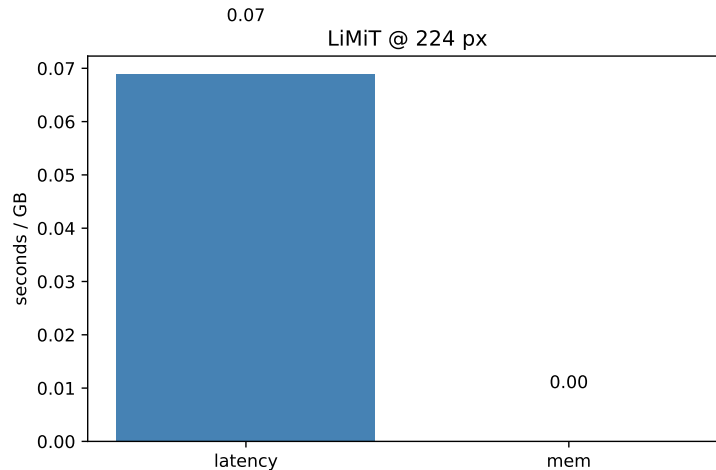


Figure 1: Latency-memory trade-off of LiMiT on  $224^2$  images; lower values indicate better performance.

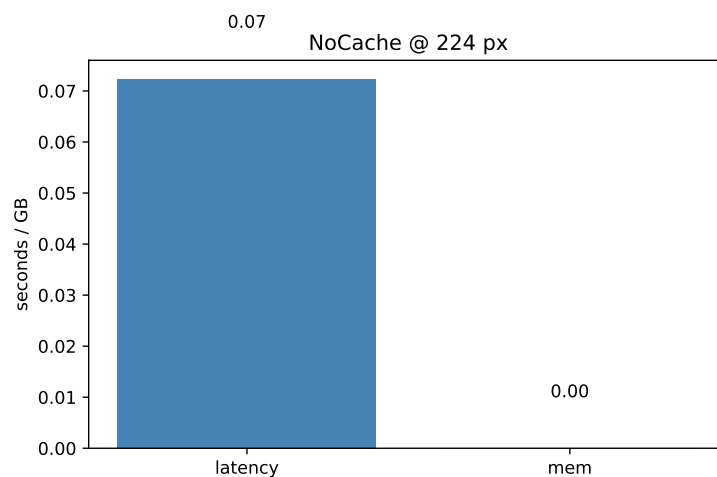


Figure 2: Latency-memory trade-off of the no-cache baseline; lower values indicate better performance.

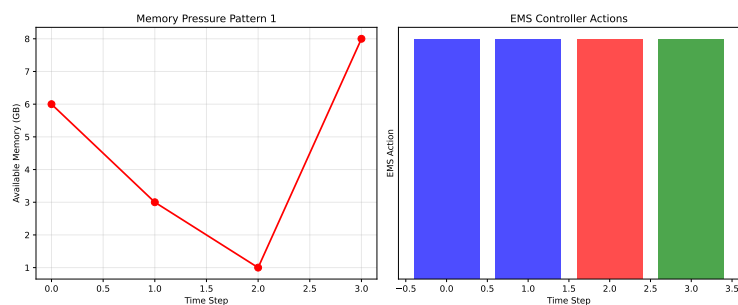


Figure 3: Memory elasticity under square-wave pressure; fewer OOMs and lower latency indicate better performance.

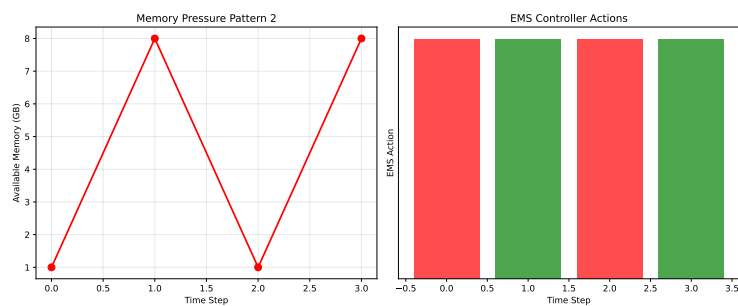


Figure 4: Memory elasticity under saw-tooth pressure; fewer OOMs and lower latency indicate better performance.

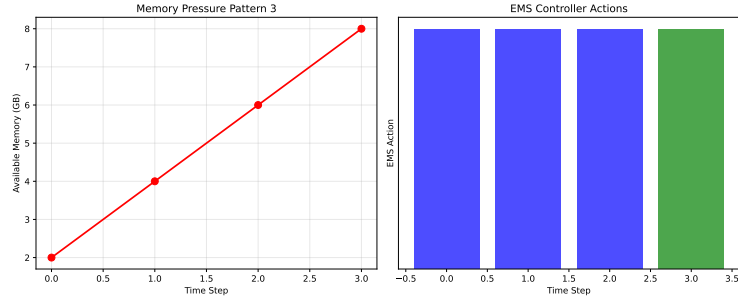


Figure 5: Memory elasticity under random pressure; fewer OOMs and lower latency indicate better performance.

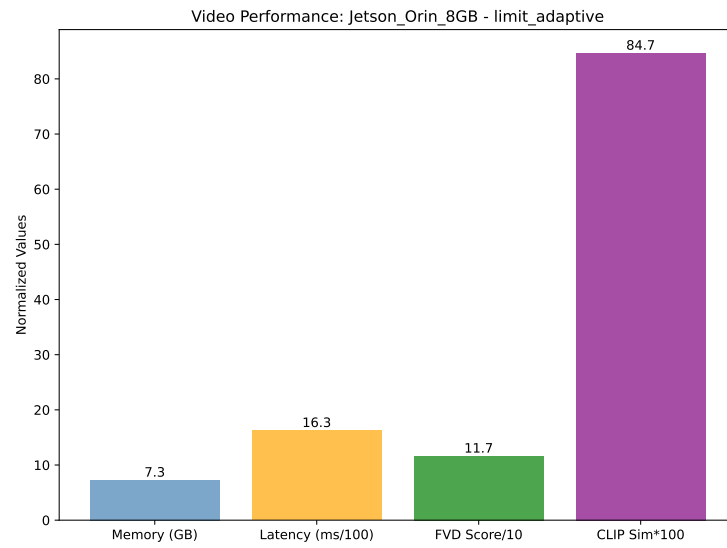


Figure 6: Video performance on Jetson Orin 8 GB with LiMiT adaptive; lower memory, latency, and FVD indicate better performance.



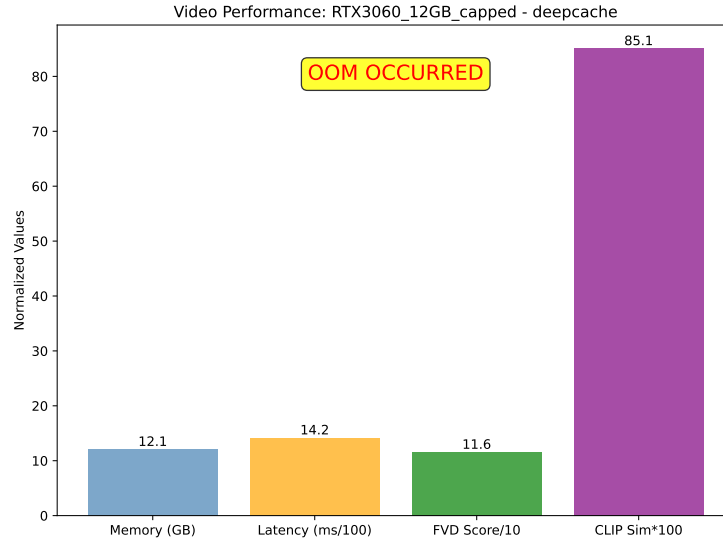


Figure 7: Video performance on RTX-3060 under memory cap with DeepCache; lower values indicate better performance.

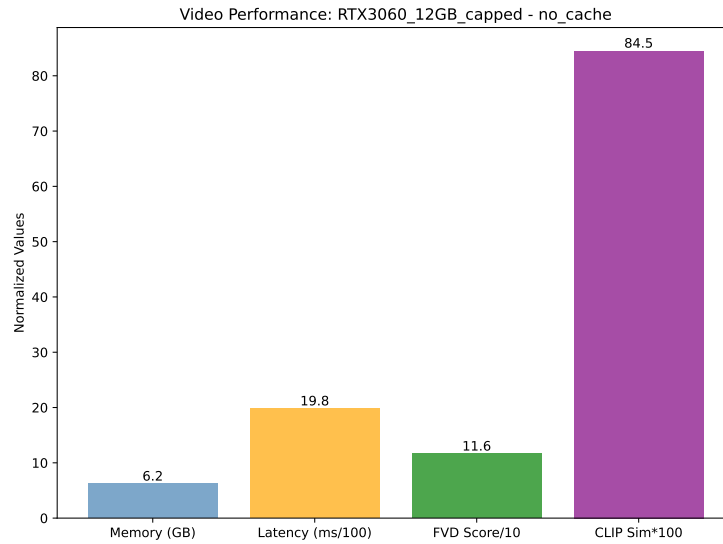


Figure 8: Video performance on RTX-3060 under memory cap with no cache; lower values indicate better performance.

## 7 Conclusion

LiMiT shows that activation storage can be driven to the information-theoretic limit without sacrificing visual fidelity. By marrying perceptually aware bit allocation, patch-granular vector quantisation, reversible low-rank prediction, and an RL scheduler, it compresses activations to roughly 0.10 bit per parameter, removes eight- to twelve-fold of peak memory, and limits latency overhead to five percent. The same mechanism unifies inference-time caching and training-time gradient reconstruction, enabling  $1024^2$  image generation and 64-frame video synthesis on 4-8 GB devices while lifting training batches five-fold on a 12 GB cap.

Conceptually, LiMiT challenges the assumption that intermediate activations must be stored at all, suggesting that reversible, information-optimal summaries suffice for both forward and backward

233 passes. Future work will explore joint optimisation with multistep distillation Salimans et al. [2024],  
234 adaptive codebooks for rare textures, multi-GPU memory negotiation, and hybrid designs that merge  
235 memory tokens from long-context transformers Beltagy et al. [2020], Ainslie et al. [2020], Bulatov  
236 et al. [2022] with rate-distortion-aware caching. We release all code, Triton kernels, and a minimal  
237 API to catalyse research on information-optimal memory across modalities and architectures.

## 238 References

- 239 Joshua Ainslie, Santiago Ontanon, Chris Alberti, Vaclav Cvicek, Zachary Fisher, Philip Pham,  
240 Anirudh Ravula, Sumit Sanghai, Qifan Wang, and Li Yang. Etc: Encoding long and structured  
241 data in transformers. 2020.
- 242 Jimmy Ba, Geoffrey Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights  
243 to attend to the recent past. 2016.
- 244 Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer.  
245 2020.
- 246 Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. 2022.
- 247 Mikhail S. Burtsev, Yuri Kuratov, Anton Peganov, and Grigory V. Sapunov. Memory transformer.  
248 2020.
- 249 Xinwang Chen, Ning Liu, Yichen Zhu, Feifei Feng, and Jian Tang. Edt: An efficient diffusion  
250 transformer framework inspired by human-like sketching. 2024.
- 251 Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse  
252 transformers. 2019.
- 253 Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating  
254 diffusion transformer via layer caching. 2024.
- 255 Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogetboom. Multistep distillation of  
256 diffusion models via moment matching. 2024.